

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-212БВ-25

Студент: Дикун Р.Г.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 07.10.26

Москва, 2026

Постановка задачи

Вариант ____.

Составить и отладить программу на языке C для ОС Windows, состоящую из двух процессов — родительского и дочернего, взаимодействующих через каналы (pipe). Родительский процесс запрашивает у пользователя имя файла, создаёт два канала и дочерний процесс, который получает имя файла в аргументе командной строки. Пользователь вводит в родительском процессе строки с целыми числами; родитель передаёт их дочернему процессу по первому каналу. Дочерний процесс делит первое число строки последовательно на все остальные и записывает результат в файл, после чего сообщает родителю статус обработки по второму каналу. При делении на ноль дочерний процесс завершается, а родительский сообщает об этом пользователю и также завершает работу. Взаимодействие с каналами, файлами и процессами реализовать средствами системных вызовов (WinAPI), без использования стандартных потоков ввода-вывода библиотеки C.

Общий метод и алгоритм решения

В программе используются системные вызовы (функции WinAPI) следующего назначения. Указаны сокращённые сигнатуры.

- **HANDLE GetStdHandle(DWORD nStdHandle);** – возвращает дескриптор стандартного потока (ввод, вывод, ошибки). Весь консольный ввод-вывод в обеих программах выполняется через эти дескрипторы.
- **BOOL CreatePipe(PHANDLE hRead, PHANDLE hWrite, LPSECURITY_ATTRIBUTES sa, DWORD size);** – создаёт анонимный канал и возвращает два дескриптора: на чтение и на запись. Если в sa задан bInheritHandle = TRUE, дескрипторы наследуемые.
- **BOOL SetHandleInformation(HANDLE h, DWORD mask, DWORD flags);** – изменяет флаги дескриптора; используется, чтобы снять флаг HANDLE_FLAG_INHERIT с концов каналов, принадлежащих только родителю.
- **BOOL CreateProcessA(..., BOOL bInheritHandles, ..., LPSTARTUPINFOA si, LPPROCESS_INFORMATION pi);** – создаёт дочерний процесс. С флагом STARTF_USESTDHANDLES в структуре STARTUPINFO дочернему процессу подставляются заданные стандартные дескрипторы, то есть концы каналов становятся его stdin и stdout.
- **BOOL ReadFile(HANDLE h, LPVOID buf, DWORD n, LPDWORD read, LPOVERLAPPED ov);** – читает данные из файла, канала или консоли; на пустом канале блокируется, а при закрытых концах записи возвращает ошибку (конец данных).
- **BOOL WriteFile(HANDLE h, LPCVOID buf, DWORD n, LPDWORD written, LPOVERLAPPED ov);** – записывает данные в файл, канал или консоль.
- **HANDLE CreateFileA(LPCSTR name, DWORD access, DWORD share, ..., DWORD disposition, ...);** – открывает или создаёт файл; с CREATE_ALWAYS

существующий файл обнуляется.

- **BOOL CloseHandle(HANDLE h);** – закрывает дескриптор. Критично для каналов: конец данных наступает, только когда закрыты все дескрипторы записывающего конца.
- **DWORD WaitForSingleObject(HANDLE h, DWORD ms);** – ожидает завершения процесса.
- **BOOL GetExitCodeProcess(HANDLE h, LPDWORD code);** – возвращает код завершения дочернего процесса.
- **DWORD GetLastError(void);** и **void ExitProcess(UINT code);** – получение кода последней ошибки и аварийное завершение процесса.

Схема взаимодействия. Канал однонаправленный, поэтому для двусторонней связи создаются два канала:

- **pipe1:** родитель (pipe1_write) → ребёнок (pipe1_read, он же stdin ребёнка) – строки с числами;
- **pipe2:** ребёнок (pipe2_write, он же stdout ребёнка) → родитель (pipe2_read) – ответы «ok», «error», «div0».

Алгоритм родительского процесса (parent.c).

- Получить стандартные дескрипторы, вывести приглашение и прочитать имя файла; убрать символы '\r', '\n'; пустое имя – ошибка.
- Создать оба канала как наследуемые (SECURITY_ATTRIBUTES.bInheritHandle = TRUE), затем снять наследование с pipe1_write и pipe2_read, чтобы ребёнок их не получил.
- Заполнить STARTUPINFO: stdin ребёнка = pipe1_read, stdout = pipe2_write, stderr = stderr родителя. Запустить *child.exe* "*имя_файла*" через CreateProcessA с bInheritHandles = TRUE.
- Сразу закрыть в родителе hThread, pipe1_read и pipe2_write – они нужны только ребёнку. Иначе признак конца данных не появится и процессы зависнут.
- В цикле читать строки со стандартного ввода (завершение – Ctrl+Z), отправлять их в pipe1 и после каждой полной строки читать ровно одну строку ответа из pipe2. «div0» – сообщить о делении на ноль и выйти из цикла; «error» – сообщить о некорректном вводе; обрыв канала – сообщить об аварийном завершении ребёнка. Строки длиннее буфера (4095 байт) пересылаются частями, а ответ ожидается только после части с символом конца строки.
- Закрыть pipe1_write (ребёнок получает конец данных) и pipe2_read, дождаться завершения ребёнка, получить код завершения; при ненулевом коде вернуть 1.

Алгоритм дочернего процесса (child.c).

- Проверить, что передан один аргумент, и открыть файл через CreateFileA (CREATE_ALWAYS).
- Читать stdin (канал) по строкам функцией read_line с динамическим буфером, который удваивается при нехватке места; NULL означает конец данных.

- Разобрать строку функцией `strtok` по пробелам, табуляциям и `'\t'`. Каждый токен переводится в `int` через `strtol` с проверкой `errno`, полного разбора токена и диапазона `int`.
- Первое число – делимое; каждое следующее – делитель. Перед делением проверяется делитель 0 (статус `div0`) и переполнение `INT_MIN / -1` (статус `error`). Деление выполняется слева направо.
- При успехе результат записывается в файл отдельной строкой (преобразование числа в текст выполнено вручную) и в `stdout` отправляется «ok»; при ошибке разбора – «error». При делении на ноль отправляется «div0», файл закрывается и процесс завершается с кодом 0.
- Когда родитель закрывает канал, `read_line` возвращает `NULL`, цикл завершается, файл закрывается.

Синхронный обмен «одна строка – один ответ» гарантирует отсутствие взаимных блокировок: процессы не пишут одновременно в заполненные каналы. Ошибки WinAPI обрабатываются на каждом вызове с выводом кода ошибки в `stderr`.

Код программы

parent.c

```
#include <windows.h>
#include <string.h>

static HANDLE std_in, std_out, std_err;

void put_str(HANDLE h, const char *s) {
    DWORD written;
    WriteFile(h, s, (DWORD)strlen(s), &written, NULL);
}

void put_ulong(HANDLE h, unsigned long n) {
    char buf[24];
    int i = sizeof(buf) - 1;

    buf[i] = '\0';
    do {
        buf[--i] = (char)('0' + n % 10);
        n /= 10;
    } while (n != 0);
    put_str(h, buf + i);
}

void error_exit(const char *message) {
    DWORD code = GetLastError();

    put_str(std_err, message);
    put_str(std_err, " failed, error code ");
    put_ulong(std_err, code);
    put_str(std_err, "\n");
    ExitProcess(1);
}

void send_data(HANDLE pipe, const char *data, DWORD size) {
    DWORD written;

    if (!WriteFile(pipe, data, size, &written, NULL) || written != size)
        error_exit("WriteFile");
}

int read_stdin_line(char *buffer, int size, int *eof) {
    int len = 0;
```

```

DWORD read_count;
char c;

while (len < size - 1) {
    if (!ReadFile(std_in, &c, 1, &read_count, NULL) || read_count == 0 ||
        c == 0x1A) {
        *eof = 1;
        break;
    }
    buffer[len++] = c;
    if (c == '\n')
        break;
}

buffer[len] = '\0';
return len;
}

int read_reply(HANDLE pipe, char *buffer, int size) {
    int len = 0;
    DWORD read_count;
    char c;

    while (len < size - 1) {
        if (!ReadFile(pipe, &c, 1, &read_count, NULL) || read_count == 0)
            return 0;
        if (c == '\n')
            break;
        if (c != '\r')
            buffer[len++] = c;
    }

    buffer[len] = '\0';
    return 1;
}

int main(void) {
    SECURITY_ATTRIBUTES sa = { sizeof(sa), NULL, TRUE };
    HANDLE pipe1_read, pipe1_write, pipe2_read, pipe2_write;
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;
    char file_name[MAX_PATH];
    char command[MAX_PATH + 16];
    char line[4096];
    char reply[64];
    DWORD exit_code;
    int eof = 0;
    int len;

    std_in = GetStdHandle(STD_INPUT_HANDLE);
    std_out = GetStdHandle(STD_OUTPUT_HANDLE);
    std_err = GetStdHandle(STD_ERROR_HANDLE);

    put_str(std_out, "Enter file name: ");
    len = read_stdin_line(file_name, sizeof(file_name), &eof);
    if (len == 0) {
        put_str(std_err, "File name was not entered\n");
        return 1;
    }
    file_name[strcspn(file_name, "\r\n")] = '\0';
    if (file_name[0] == '\0') {
        put_str(std_err, "File name is empty\n");
        return 1;
    }

    if (!CreatePipe(&pipe1_read, &pipe1_write, &sa, 0))
        error_exit("CreatePipe (pipe1)");
    if (!CreatePipe(&pipe2_read, &pipe2_write, &sa, 0))
        error_exit("CreatePipe (pipe2)");
    if (!SetHandleInformation(pipe1_write, HANDLE_FLAG_INHERIT, 0))
        error_exit("SetHandleInformation (pipe1)");
    if (!SetHandleInformation(pipe2_read, HANDLE_FLAG_INHERIT, 0))
        error_exit("SetHandleInformation (pipe2)");

```

```

ZeroMemory(&si, sizeof(si));
si.cb = sizeof(si);
si.dwFlags = STARTF_USESTDHANDLES;
si.hStdInput = pipe1_read;
si.hStdOutput = pipe2_write;
si.hStdError = std_err;
ZeroMemory(&pi, sizeof(pi));

lstrcpyA(command, "child.exe \"");
lstrcatA(command, file_name);
lstrcatA(command, "\"");
if (!CreateProcessA(NULL, command, NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi))
    error_exit("CreateProcess");

CloseHandle(pi.hThread);
CloseHandle(pipe1_read);
CloseHandle(pipe2_write);

eof = 0;
put_str(std_out, "Enter numbers (Ctrl+Z to finish):\n");
while ((len = read_stdin_line(line, sizeof(line), &eof)) > 0) {
    int line_ended = line[len - 1] == '\n';

    send_data(pipe1_write, line, (DWORD)len);
    if (!line_ended && !eof)
        continue;
    if (!line_ended)
        send_data(pipe1_write, "\n", 1);

    if (!read_reply(pipe2_read, reply, sizeof(reply))) {
        put_str(std_err, "Child process terminated unexpectedly\n");
        break;
    }
    if (strcmp(reply, "div0") == 0) {
        put_str(std_out, "Division by zero. Terminating.\n");
        break;
    }
    if (strcmp(reply, "error") == 0)
        put_str(std_out, "Invalid input, expected integers\n");
}

CloseHandle(pipe1_write);
CloseHandle(pipe2_read);

if (WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_FAILED)
    error_exit("WaitForSingleObject");
if (!GetExitCodeProcess(pi.hProcess, &exit_code))
    error_exit("GetExitCodeProcess");
CloseHandle(pi.hProcess);

if (exit_code != 0) {
    put_str(std_err, "Child process exited with code ");
    put_ulong(std_err, exit_code);
    put_str(std_err, "\n");
    return 1;
}
return 0;
}

```

child.c

```

#include <windows.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>

enum { STATUS_OK,
        STATUS_ERROR,
        STATUS_DIV0 };

static HANDLE std_in, std_out, std_err;

void put_str(HANDLE h, const char *s) {

```

```

    DWORD written;
    WriteFile(h, s, (DWORD)strlen(s), &written, NULL);
}

void fail(const char *message) {
    DWORD code = GetLastError();
    char buf[24];
    int i = sizeof(buf) - 1;

    buf[i] = '\\0';
    do {
        buf[--i] = (char)('0' + code % 10);
        code /= 10;
    } while (code != 0);

    put_str(std_err, message);
    put_str(std_err, " failed, error code ");
    put_str(std_err, buf + i);
    put_str(std_err, "\\n");
}

int write_int_line(HANDLE h, int value) {
    char buf[16];
    int i = sizeof(buf) - 1;
    unsigned int n = value < 0 ? 0u - (unsigned int)value : (unsigned int)value;
    DWORD written, size;

    buf[i] = '\\n';
    do {
        buf[--i] = (char)('0' + n % 10);
        n /= 10;
    } while (n != 0);
    if (value < 0)
        buf[--i] = '-';

    size = (DWORD)(sizeof(buf) - i);
    return WriteFile(h, buf + i, size, &written, NULL) && written == size;
}

int read_char(void) {
    char c;
    DWORD read_count;

    if (!ReadFile(std_in, &c, 1, &read_count, NULL) || read_count == 0)
        return -1;
    return (unsigned char)c;
}

char *read_line(void) {
    size_t size = 64, len = 0;
    char *line = malloc(size);
    int c;

    if (line == NULL) {
        put_str(std_err, "malloc failed\\n");
        exit(1);
    }

    while ((c = read_char()) != -1 && c != '\\n') {
        if (len + 1 == size) {
            char *tmp;
            size *= 2;
            tmp = realloc(line, size);
            if (tmp == NULL) {
                put_str(std_err, "realloc failed\\n");
                free(line);
                exit(1);
            }
            line = tmp;
        }
        line[len++] = (char)c;
    }

    if (c == -1 && len == 0) {

```

```

        free(line);
        return NULL;
    }

    line[len] = '\0';
    return line;
}

int parse_int(const char *token, int *value) {
    char *end;
    long number;

    errno = 0;
    number = strtol(token, &end, 10);
    if (errno != 0 || *end != '\0' || number < INT_MIN || number > INT_MAX)
        return 0;

    *value = (int)number;
    return 1;
}

int divide_line(char *line, int *result) {
    const char *delims = " \t\r";
    char *token = strtok(line, delims);
    int divisor;

    if (token == NULL || !parse_int(token, result))
        return STATUS_ERROR;

    while ((token = strtok(NULL, delims)) != NULL) {
        if (!parse_int(token, &divisor))
            return STATUS_ERROR;
        if (divisor == 0)
            return STATUS_DIV0;
        if (*result == INT_MIN && divisor == -1)
            return STATUS_ERROR;
        *result /= divisor;
    }

    return STATUS_OK;
}

int main(int argc, char *argv[]) {
    HANDLE out;
    char *line;

    std_in = GetStdHandle(STD_INPUT_HANDLE);
    std_out = GetStdHandle(STD_OUTPUT_HANDLE);
    std_err = GetStdHandle(STD_ERROR_HANDLE);

    if (argc != 2) {
        put_str(std_err, "Usage: ");
        put_str(std_err, argv[0]);
        put_str(std_err, " <file>\n");
        return 1;
    }

    out = CreateFileA(argv[1], GENERIC_WRITE, FILE_SHARE_READ, NULL,
                     CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (out == INVALID_HANDLE_VALUE) {
        fail("CreateFile");
        return 1;
    }

    while ((line = read_line()) != NULL) {
        int result;
        int status = divide_line(line, &result);
        free(line);

        if (status == STATUS_DIV0) {
            put_str(std_out, "div0\n");
            CloseHandle(out);
            return 0;
        }
    }
}

```

```

        if (status == STATUS_OK) {
            if (!write_int_line(out, result)) {
                fail("write to file");
                CloseHandle(out);
                return 1;
            }
            put_str(std_out, "ok\n");
        } else {
            put_str(std_out, "error\n");
        }
    }

    if (!CloseHandle(out)) {
        fail("CloseHandle");
        return 1;
    }
    return 0;
}

```

Протокол работы программы

Сборка:

```

$ gcc parent.c -o parent.exe
$ gcc child.c -o child.exe

```

Тест 1. Корректные данные, некорректный ввод и деление на ноль (ввод подаётся со стандартного ввода; после «9 3 0» ввод прекращается, строка «55 1» не обрабатывается):

```

$ printf 'out1.txt\n100 5 2\n-20 4\n7\n12 abc\n-2147483648 -1\n9 3 0\n55 1\n' | ./parent.exe
Enter file name: Enter numbers (Ctrl+Z to finish):
Invalid input, expected integers
Invalid input, expected integers
Division by zero. Terminating.
$ echo $?
0
$ cat out1.txt
10
-5
7

```

Строки «100 5 2», «-20 4» и «7» дали 10, -5 и 7 (7 – единственное число без делителей). «12 abc» отвергнута из-за нечислового токена, «-2147483648 -1» – из-за переполнения. В файл попадают только успешные результаты.

Тест 2. Целочисленное деление с усечением:

```

$ printf 'out2.txt\n8 2\n1 2 3\n' | ./parent.exe
Enter file name: Enter numbers (Ctrl+Z to finish):
$ cat out2.txt
4
0

```

$8 / 2 = 4$; $1 / 2 / 3 = (1 / 2) / 3 = 0 / 3 = 0$, так как деление целочисленное.

Тест 3. Недопустимый путь к файлу:

```

$ printf '%s\n' 'nodir\x.txt' '5 1' | ./parent.exe
Enter file name: Enter numbers (Ctrl+Z to finish):
CreateFile failed, error code 3
Child process terminated unexpectedly
Child process exited with code 1
$ echo $?
1

```

Дочерний процесс не смог создать файл (код 3 – ERROR_PATH_NOT_FOUND), сообщил об этом и завершился с кодом 1; родитель обнаружил обрыв канала и вернул

ненулевой код.

Тест 4. Пустое имя файла:

```
$ printf '\n' | ./parent.exe
Enter file name: File name is empty
$ echo $?
1
```

Вывод

В ходе работы были изучены средства взаимодействия процессов в Windows: создание процессов (CreateProcess), анонимные каналы (CreatePipe), наследование и закрытие дескрипторов, перенаправление стандартных потоков дочернего процесса и работа с файлами через WriteFile и CreateFile. Реализован синхронный протокол обмена между родительским и дочерним процессами по двум однонаправленным каналам.

Основная сложность – корректное управление дескрипторами: если родитель оставляет открытыми лишние концы каналов или ребёнок наследует чужие, признак конца данных не наступает и программа зависает. Её удалось избежать, сняв флаг наследования с концов, принадлежащих родителю, и закрыв ненужные дескрипторы сразу после создания процесса. Также учтены особые случаи: деление на ноль, переполнение INT_MIN / -1, строки произвольной длины и обрыв канала.